

Inteligencia Artificial en Ciberseguridad

Guía Técnica Detallada con Python

Guía Técnica de Implementación

Mayo 2026

Índice general

1. Introducción a la Inteligencia Artificial en Ciberseguridad	3
1.1. Panorama general	3
1.2. Tecnologías subyacentes	3
1.3. Por qué Python	4
2. Configuración del Entorno de Desarrollo Python	5
2.1. Requisitos previos	5
2.2. Creación del entorno virtual	5
2.3. Instalación de bibliotecas	5
2.4. Verificación del entorno	6
3. IA para la Detección de Amenazas	7
3.1. Introducción	7
3.2. Recolección y preprocesamiento de datos	7
3.2.1. Carga de datos de tráfico de red	7
3.2.2. Preprocesamiento y normalización	7
3.3. Detección de anomalías con Isolation Forest	8
3.4. Detección de intrusiones con Autoencoder (Deep Learning)	10
4. IA para la Detección de Malware	13
4.1. Limitaciones del enfoque basado en firmas	13
4.2. Extracción de características de archivos PE	13
4.3. Construcción del dataset de malware	15
4.4. Modelo de clasificación de malware	16
4.4.1. Árbol de decisión	16
4.4.2. Random Forest con manejo de clases desbalanceadas	17
5. IA para la Automatización de la Respuesta a Incidentes	19
5.1. Introducción	19
5.2. Triage automático de incidentes	19
5.2.1. Clasificación de severidad con SVM	19
5.3. Orquestación de respuestas con Python	20
6. Análisis de Comportamiento de Usuarios (UBA)	23
6.1. Fundamentos del UBA	23
6.2. Construcción de perfiles de usuario	23

7. Explicabilidad de Modelos de IA (XAI)	26
7.1. Importancia en ciberseguridad	26
7.2. SHAP: SHapley Additive exPlanations	26
8. Ataques Adversariales contra Modelos de IA	28
8.1. Amenazas a los modelos de IA	28
8.2. Defensa: entrenamiento adversarial	28
9. Desafíos y Buenas Prácticas	30
9.1. Desafíos en la implementación	30
9.2. Buenas prácticas de desarrollo seguro	31
9.3. Monitoreo de deriva de datos en producción	31
10.Integración Completa: Pipeline de Seguridad con IA	33
10.1. Arquitectura del sistema	33
10.2. Pipeline completo en Python	33
11.Resumen y Perspectivas Futuras	36
11.1. Lo aprendido	36
11.2. Hoja de ruta recomendada	36

Capítulo 1

Introducción a la Inteligencia Artificial en Ciberseguridad

1.1. Panorama general

La aplicación de la Inteligencia Artificial (IA) en ciberseguridad está transformando la manera en que las organizaciones protegen sus sistemas, datos y usuarios. La IA, especialmente cuando se combina con Python, ofrece herramientas poderosas para detectar, analizar y responder a amenazas cibernéticas en tiempo real.

¿Qué entendemos por IA en ciberseguridad?

La IA en ciberseguridad hace referencia al uso de sistemas inteligentes capaces de:

- Aprender de forma autónoma a partir de grandes volúmenes de datos.
- Detectar patrones anómalos y tomar decisiones que refuercen la postura de seguridad.
- Automatizar la respuesta ante incidentes, reduciendo la intervención humana.

1.2. Tecnologías subyacentes

Los sistemas de seguridad basados en IA aprovechan tres grandes disciplinas:

Aprendizaje Automático (ML) Algoritmos que aprenden de los datos sin ser programados explícitamente para cada tarea. Ejemplos: árboles de decisión, bosques aleatorios, SVM.

Aprendizaje Profundo (DL) Redes neuronales con múltiples capas que extraen representaciones de alto nivel. Especialmente útiles en análisis de malware y detección de intrusiones.

Procesamiento de Lenguaje Natural (NLP) Permite analizar texto de logs, informes de incidentes y comunicaciones para identificar amenazas semánticas.

1.3. Por qué Python

Python ocupa un lugar central en el desarrollo de IA y ML gracias a:

- Legibilidad y facilidad de aprendizaje.
- Comunidad activa y ecosistema maduro de bibliotecas (`scikit-learn`, `TensorFlow`, `PyTorch`, `Keras`).
- Simplicidad de integración con APIs de plataformas de seguridad.
- Herramientas especializadas en análisis forense y análisis de binarios (`pefile`, `yara-python`, `capstone`).

Capítulo 2

Configuración del Entorno de Desarrollo Python

2.1. Requisitos previos

Antes de implementar soluciones de seguridad basadas en IA es indispensable preparar un entorno de desarrollo reproducible y aislado.

Versión mínima requerida

Se recomienda Python **3.9 o superior**. Versiones anteriores a 3.6 no son compatibles con varias de las bibliotecas utilizadas en esta guía.

2.2. Creación del entorno virtual

```
1 # Crear el entorno
2 python3 -m venv ai_security_env
3
4 # Activar en Linux / macOS
5 source ai_security_env/bin/activate
6
7 # Activar en Windows
8 ai_security_env\Scripts\activate
```

Listing 2.1: Crear y activar un entorno virtual

2.3. Instalación de bibliotecas

```
1 pip install numpy pandas scikit-learn tensorflow keras \
2 matplotlib seaborn pefile requests joblib \
3 imbalanced-learn shap lime
```

Listing 2.2: Instalación de dependencias principales

Cuadro 2.1: Bibliotecas principales y su función

Biblioteca	Función en seguridad
NumPy	Operaciones matriciales sobre vectores de características
pandas	Carga y preprocesamiento de logs y datasets
scikit-learn	Algoritmos clásicos de ML (SVM, árboles, bosques)
TensorFlow	Modelos de DL para detección de intrusiones
Keras	API de alto nivel sobre TensorFlow
pefile	Análisis de ejecutables PE para detección de malware
shap	Explicabilidad de modelos (XAI)
imbalanced-learn	Manejo de clases desbalanceadas en datasets de seguridad

2.4. Verificación del entorno

```

1 import sys
2 import importlib
3
4 REQUIRED = [
5     "numpy", "pandas", "sklearn", "tensorflow",
6     "keras", "matplotlib", "seaborn", "pefile", "requests"
7 ]
8
9 print(f"Python {sys.version}\n{'='*40}")
10 for pkg in REQUIRED:
11     try:
12         mod = importlib.import_module(pkg)
13         version = getattr(mod, "__version__", "OK")
14         print(f" [OK]  {pkg:<18} {version}")
15     except ImportError:
16         print(f" [FALTA] {pkg}")

```

Listing 2.3: Script de verificación del entorno

Capítulo 3

IA para la Detección de Amenazas

3.1. Introducción

La detección de amenazas es una de las aplicaciones más efectivas de la IA en seguridad. Los modelos de IA pueden analizar tráfico de red, comportamiento de usuarios y registros de sistema para identificar patrones anómalos que indiquen una posible brecha de seguridad.

3.2. Recolección y preprocesamiento de datos

3.2.1. Carga de datos de tráfico de red

```
1 import pandas as pd
2 import numpy as np
3 import matplotlib.pyplot as plt
4 import seaborn as sns
5
6 # Cargar datos de tráfico de red
7 # El dataset debe incluir columnas como: bytes_sent, bytes_recv,
8 # duration, protocol, src_port, dst_port, etc.
9 data = pd.read_csv('network_traffic.csv')
10
11 print("=== Información del dataset ===")
12 print(data.info())
13 print("\n=== Primeras filas ===")
14 print(data.head())
15 print("\n=== Estadísticas descriptivas ===")
16 print(data.describe())
17
18 # Verificar valores nulos
19 print("\n=== Valores nulos por columna ===")
20 print(data.isnull().sum())
```

Listing 3.1: Carga y exploración de datos de tráfico de red

3.2.2. Preprocesamiento y normalización


```

1 import pandas as pd
2 import numpy as np
3 from sklearn.preprocessing import MinMaxScaler, LabelEncoder
4
5 # Cargar datos
6 data = pd.read_csv('network_traffic.csv')
7
8 # 1. Eliminar filas con valores nulos
9 data = data.dropna()
10
11 # 2. Codificar variables categóricas (e.g., protocolo)
12 le = LabelEncoder()
13 if 'protocol' in data.columns:
14     data['protocol'] = le.fit_transform(data['protocol'])
15
16 # 3. Seleccionar características numéricas relevantes
17 feature_cols = ['bytes_sent', 'bytes_recv', 'duration',
18                 'src_port', 'dst_port', 'protocol']
19 features = data[feature_cols].copy()
20
21 # 4. Normalizar entre 0 y 1
22 scaler = MinMaxScaler()
23 scaled_data = scaler.fit_transform(features)
24 scaled_df = pd.DataFrame(scaled_data, columns=feature_cols)
25
26 print("Datos escalados (primeras 5 filas):")
27 print(scaled_df.head())
28
29 # 5. Visualizar distribución de bytes enviados
30 plt.figure(figsize=(10, 4))
31 plt.subplot(1, 2, 1)
32 plt.hist(features['bytes_sent'], bins=50, color='steelblue',
33          alpha=0.7)
34 plt.title('Distribución original - bytes_sent')
35 plt.subplot(1, 2, 2)
36 plt.hist(scaled_df['bytes_sent'], bins=50, color='darkorange',
37          alpha=0.7)
38 plt.title('Distribución normalizada - bytes_sent')
39 plt.tight_layout()
40 plt.savefig('network_distribution.png', dpi=150)
41 plt.show()

```

Listing 3.2: Preprocesamiento completo del dataset de tráfico

3.3. Detección de anomalías con Isolation Forest

El algoritmo *Isolation Forest* aísla anomalías particionando el espacio de características de manera recursiva. Las anomalías son puntos que se aíslan con pocas particiones, lo que los hace fáciles de separar del resto.

```

1 import pandas as pd
2 import numpy as np
3 import matplotlib.pyplot as plt
4 from sklearn.preprocessing import MinMaxScaler
5 from sklearn.ensemble import IsolationForest
6 from sklearn.metrics import classification_report
7
8 # 1. Cargar y escalar datos
9
10 data = pd.read_csv('network_traffic.csv').dropna()
11 feature_cols = ['bytes_sent', 'bytes_recv', 'duration',
12                 'src_port']
13 scaler = MinMaxScaler()
14 scaled_data = scaler.fit_transform(data[feature_cols])
15
16 # 2. Entrenar Isolation Forest
17
18 # contamination: fracción esperada de anomalías (5%)
19 model = IsolationForest(
20     n_estimators=200,
21     contamination=0.05,
22     random_state=42,
23     n_jobs=-1 # usar todos los núcleos disponibles
24 )
25 model.fit(scaled_data)
26
27 # 3. Predicción
28
29 # 1 = normal
30 # -1 = anomalía
31 predictions = model.predict(scaled_data)
32 scores = model.decision_function(scaled_data) # puntuación de
33         anomalía
34
35 data['anomaly'] = predictions
36 data['anomaly_score'] = scores
37
38 # 4. Resultados
39
40 anomalous = data[data['anomaly'] == -1]
41 normal = data[data['anomaly'] == 1]
42
43 print(f"Total de muestras : {len(data)}")
44 print(f"Muestras normales : {len(normal)}")
45 print(f"Anomalías detectadas: {len(anomalous)}")
46 print("\nTop 10 anomalías (menor puntuación = más anómalo):")
47 print(anomalous.sort_values('anomaly_score').head(10))
48
49 # 5. Visualización
50
51 plt.figure(figsize=(10, 5))

```

```

45 plt.scatter(range(len(normal)),
46             normal['bytes_sent'], s=5, c='steelblue',
47             alpha=0.4, label='Normal')
48 plt.scatter(anomalous.index,
49             anomalous['bytes_sent'], s=20, c='red',
50             alpha=0.8, label='Anomalía')
51 plt.xlabel('Índice de muestra')
52 plt.ylabel('bytes_sent')
53 plt.title('Anomalías detectadas en tráfico de red')
54 plt.legend()
55 plt.tight_layout()
56 plt.savefig('anomaly_detection.png', dpi=150)
57 plt.show()

```

Listing 3.3: Modelo de detección de anomalías con Isolation Forest

3.4. Detección de intrusiones con Autoencoder (Deep Learning)

Un *autoencoder* aprende a reconstruir entradas normales. Cuando procesa datos anómalos, el error de reconstrucción es alto, permitiendo detectar intrusiones no vistas durante el entrenamiento.

```

1  import numpy as np
2  import pandas as pd
3  import matplotlib.pyplot as plt
4  from sklearn.preprocessing import MinMaxScaler
5  from sklearn.model_selection import train_test_split
6  from tensorflow import keras
7  from tensorflow.keras import layers
8
9  # 1. Preparar datos (solo tráfico normal para
   #   entrenamiento)
10 data = pd.read_csv('network_traffic.csv').dropna()
11 feature_cols = ['bytes_sent', 'bytes_recv', 'duration',
   'src_port']
12
13 scaler = MinMaxScaler()
14 X = scaler.fit_transform(data[feature_cols])
15
16 # Dividir: 80% entrenamiento (solo normales), 20% prueba
17 X_train, X_test = train_test_split(X, test_size=0.2,
   random_state=42)
18
19 # 2. Definir arquitectura del autoencoder
20
21 input_dim = X.shape[1]
22
23 def build_autoencoder(input_dim, encoding_dim=8):
   """Constuye un autoencoder simétrico."""

```

```

24     # Codificador
25     inp = keras.Input(shape=(input_dim,))
26     encoded = layers.Dense(16, activation='relu')(inp)
27     encoded = layers.Dense(encoding_dim,
28                             activation='relu')(encoded)
29     # Decodificador
30     decoded = layers.Dense(16, activation='relu')(encoded)
31     decoded = layers.Dense(input_dim,
32                             activation='sigmoid')(decoded)
33
34     autoencoder = keras.Model(inp, decoded, name='autoencoder')
35     encoder      = keras.Model(inp, encoded, name='encoder')
36     return autoencoder, encoder
37
38 autoencoder, encoder = build_autoencoder(input_dim)
39 autoencoder.compile(optimizer='adam', loss='mse')
40 autoencoder.summary()
41
42 # 3. Entrenar
43
44 history = autoencoder.fit(
45     X_train, X_train,
46     epochs=50,
47     batch_size=64,
48     validation_split=0.15,
49     verbose=1
50 )
51
52 # 4. Calcular error de reconstrucción
53
54 X_pred = autoencoder.predict(X_test)
55 mse_err = np.mean(np.power(X_test - X_pred, 2), axis=1)
56
57 # Umbral: percentil 95 del error en datos de entrenamiento
58 X_train_pred = autoencoder.predict(X_train)
59 train_mse = np.mean(np.power(X_train - X_train_pred, 2),
60                     axis=1)
61 threshold = np.percentile(train_mse, 95)
62 print(f"Umbral de detección: {threshold:.6f}")
63
64 # Clasificar anomalías
65 anomalies_detected = mse_err > threshold
66 print(f"Anomalías detectadas en test:
67       {anomalies_detected.sum()}")
68
69 # 5. Curva de pérdida
70
71 plt.figure(figsize=(8, 4))
72 plt.plot(history.history['loss'], label='Pérdida
73         entrenamiento')
74 plt.plot(history.history['val_loss'], label='Pérdida validación')

```

```
67 plt.title('Curva de aprendizaje del Autoencoder')
68 plt.xlabel('Época')
69 plt.ylabel('MSE')
70 plt.legend()
71 plt.tight_layout()
72 plt.savefig('autoencoder_loss.png', dpi=150)
73 plt.show()
```

Listing 3.4: Autoencoder para detección de intrusiones

Capítulo 4

IA para la Detección de Malware

4.1. Limitaciones del enfoque basado en firmas

Los sistemas antivirus tradicionales dependen de la detección basada en firmas, lo que los hace ineficaces frente a malware nuevo o polimórfico. La IA puede detectar malware analizando el **comportamiento** y las **características estructurales** de los archivos.

4.2. Extracción de características de archivos PE

Los archivos ejecutables portátiles (PE) contienen metadatos ricos: número de secciones, punto de entrada, tablas de importaciones/exportaciones, entropía, etc.

```
1 pip install pefile
```

Listing 4.1: Instalar biblioteca pefile

```
1 import pefile
2 import math
3 import os
4
5 def calcular_entropia(data: bytes) -> float:
6     """Calcula la entropía de Shannon de un bloque de bytes."""
7     if not data:
8         return 0.0
9     freq = [data.count(bytes([b])) / len(data) for b in
10             range(256)]
11     return -sum(p * math.log2(p) for p in freq if p > 0)
12
13 def extraer_caracteristicas_pe(ruta_archivo: str) -> dict:
14     """
15     Extrae características relevantes de un ejecutable PE.
16     Retorna un diccionario con las características o None si
17     falla.
18     """
19     try:
20         pe = pefile.PE(ruta_archivo)
21     except pefile.PEFormatError:
22         print(f"[!] No es un PE válido: {ruta_archivo}")
23         return None
```

```

22
23     caracteristicas = {}
24
25     #           Cabecera opcional
26
27     caracteristicas['entry_point'] =
28         pe.OPTIONAL_HEADER.AddressOfEntryPoint
29     caracteristicas['image_base'] =
30         pe.OPTIONAL_HEADER.ImageBase
31     caracteristicas['size_of_image'] =
32         pe.OPTIONAL_HEADER.SizeOfImage
33     caracteristicas['size_code_section'] =
34         pe.OPTIONAL_HEADER.SizeOfCode
35     caracteristicas['dll_flag'] =
36         pe.OPTIONAL_HEADER.DllCharacteristics
37
38     #           Secciones
39
40     caracteristicas['num_sections'] = len(pe.sections)
41     entropias = [calcular_entropia(sec.get_data()) for sec in
42         pe.sections]
43     caracteristicas['entropia_max'] = max(entropias) if
44         entropias else 0.0
45     caracteristicas['entropia_media'] = (
46         sum(entropias) / len(entropias) if entropias else 0.0
47     )
48
49     #           Importaciones / exportaciones
50
51     if hasattr(pe, 'DIRECTORY_ENTRY_IMPORT'):
52         caracteristicas['num_importaciones'] = sum(
53             len(entry.imports)
54             for entry in pe.DIRECTORY_ENTRY_IMPORT
55         )
56         caracteristicas['num_dlls_importadas'] =
57             len(pe.DIRECTORY_ENTRY_IMPORT)
58     else:
59         caracteristicas['num_importaciones'] = 0
60         caracteristicas['num_dlls_importadas'] = 0
61
62     if hasattr(pe, 'DIRECTORY_ENTRY_EXPORT'):
63         caracteristicas['num_exportaciones'] = len(
64             pe.DIRECTORY_ENTRY_EXPORT.symbols
65         )
66     else:
67         caracteristicas['num_exportaciones'] = 0
68
69     #           Tamaño del archivo
70
71     caracteristicas['file_size'] = os.path.getsize(ruta_archivo)

```

```

61     pe.close()
62     return características
63
64     # Uso de ejemplo
65
66     ruta = 'muestra.exe'
67     feats = extraer_caracteristicas_pe(ruta)
68     if feats:
69         for k, v in feats.items():
70             print(f"    {k:<28}: {v}")

```

Listing 4.2: Extracción de características de un archivo PE

4.3. Construcción del dataset de malware

```

1  import os
2  import pandas as pd
3
4  def construir_dataset(directorios: dict) -> pd.DataFrame:
5      """
6      directorios: {'benign': '/ruta/benign', 'malicious':
7                  '/ruta/malicious'}
8      Retorna un DataFrame con características y etiqueta.
9      """
10     registros = []
11     for etiqueta, directorio in directorios.items():
12         valor_label = 0 if etiqueta == 'benign' else 1
13         for archivo in os.listdir(directorio):
14             ruta_completa = os.path.join(directorio, archivo)
15             feats = extraer_caracteristicas_pe(ruta_completa)
16             if feats:
17                 feats['label'] = valor_label
18                 registros.append(feats)
19
20     df = pd.DataFrame(registros)
21     print(f"Dataset: {len(df)} muestras | "
22           f"Benign={df['label'].eq(0).sum()} | "
23           f"Malicious={df['label'].eq(1).sum()}")
24     return df
25
26 directorios = {
27     'benign': '/datasets/benign',
28     'malicious': '/datasets/malicious',
29 }
30 df = construir_dataset(directorios)
31 df.to_csv('file_features.csv', index=False)

```

Listing 4.3: Generar dataset con etiquetas benign/malicious

4.4. Modelo de clasificación de malware

4.4.1. Árbol de decisión

```
1 import pandas as pd
2 from sklearn.tree import DecisionTreeClassifier, export_text
3 from sklearn.model_selection import train_test_split,
4     cross_val_score
5 from sklearn.metrics import (classification_report,
6     confusion_matrix,
7     ConfusionMatrixDisplay)
8 import matplotlib.pyplot as plt
9
10 # 1. Cargar datos
11
12 df = pd.read_csv('file_features.csv').dropna()
13 X = df.drop('label', axis=1)
14 y = df['label']
15
16 # 2. Dividir en entrenamiento y prueba
17
18 X_train, X_test, y_train, y_test = train_test_split(
19     X, y, test_size=0.2, random_state=42, stratify=y
20 )
21
22 # 3. Entrenar
23
24 clf = DecisionTreeClassifier(max_depth=10, random_state=42)
25 clf.fit(X_train, y_train)
26
27 # 4. Evaluación
28
29 y_pred = clf.predict(X_test)
30 print("=== Reporte de clasificación ===")
31 print(classification_report(y_test, y_pred,
32     target_names=['Benign',
33     'Malicious']))
34
35 # Validación cruzada 5-fold
36 cv_scores = cross_val_score(clf, X, y, cv=5, scoring='f1')
37 print(f"F1 CV (5-fold): {cv_scores.mean():.4f}
38     {cv_scores.std():.4f}")
39
40 # 5. Matriz de confusión
41
42 cm = confusion_matrix(y_test, y_pred)
43 disp = ConfusionMatrixDisplay(cm,
44     display_labels=['Benign', 'Malicious'])
45 disp.plot(cmap='Blues')
46 plt.title('Matriz de Confusión      Árbol de Decisión')
47 plt.tight_layout()
```

```

38 plt.savefig('confusion_matrix_dt.png', dpi=150)
39 plt.show()
40
41 #          6. Importancia de características
42
43 importancias = pd.Series(clf.feature_importances_,
44                          index=X.columns)
45 top10 = importancias.nlargest(10)
46 top10.plot(kind='barh', color='steelblue', figsize=(8, 5))
47 plt.title('Top 10 características más importantes')
48 plt.xlabel('Importancia')
49 plt.tight_layout()
50 plt.savefig('feature_importance.png', dpi=150)
51 plt.show()

```

Listing 4.4: Clasificador de malware con árbol de decisión

4.4.2. Random Forest con manejo de clases desbalanceadas

```

1  import pandas as pd
2  from sklearn.ensemble import RandomForestClassifier
3  from sklearn.model_selection import train_test_split
4  from sklearn.metrics import classification_report
5  from imblearn.over_sampling import SMOTE
6
7  #          1. Cargar datos
8
9  df = pd.read_csv('file_features.csv').dropna()
10 X, y = df.drop('label', axis=1), df['label']
11
12 #          2. Balancear clases con SMOTE (Synthetic Minority
13     Oversampling)
14 X_train, X_test, y_train, y_test = train_test_split(
15     X, y, test_size=0.2, random_state=42, stratify=y
16 )
17 smote = SMOTE(random_state=42)
18 X_train_res, y_train_res = smote.fit_resample(X_train, y_train)
19 print(f"Antes SMOTE: {y_train.value_counts().to_dict()}")
20 print(f"Después SMOTE: {y_train_res.value_counts().to_dict()}")
21
22 #          3. Entrenar Random Forest
23
24 rf = RandomForestClassifier(
25     n_estimators=300,
26     max_depth=None,
27     n_jobs=-1,
28     random_state=42
29 )
30 rf.fit(X_train_res, y_train_res)

```

```

29 # 4. Evaluación
30 y_pred = rf.predict(X_test)
31 print(classification_report(y_test, y_pred,
32                             target_names=['Benign',
                                             'Malicious']))

```

Listing 4.5: Random Forest con SMOTE para clases desbalanceadas

Capítulo 5

IA para la Automatización de la Respuesta a Incidentes

5.1. Introducción

La IA puede integrarse en sistemas SIEM (*Security Information and Event Management*) para agilizar la identificación, contención y resolución de incidentes de seguridad, reduciendo el tiempo medio de respuesta (MTTR).

5.2. Triage automático de incidentes

5.2.1. Clasificación de severidad con SVM

```
1 import pandas as pd
2 import numpy as np
3 from sklearn.svm import SVC
4 from sklearn.pipeline import Pipeline
5 from sklearn.preprocessing import StandardScaler
6 from sklearn.model_selection import train_test_split
7 from sklearn.metrics import classification_report
8
9 # 1. Cargar datos de incidentes
10
11 # Columnas esperadas: num_hosts_afectados, tipo_evento_cod,
12 # bytes_exfiltrados, duracion_seg, privilegios_elevados, etc.
13 # Etiqueta: 0=bajo, 1=medio, 2=alto, 3=crítico
14 incidents = pd.read_csv('incident_data.csv').dropna()
15 X = incidents.drop('severity', axis=1)
16 y = incidents['severity']
17
18 X_train, X_test, y_train, y_test = train_test_split(
19     X, y, test_size=0.2, stratify=y, random_state=42
20 )
21
22 # 2. Pipeline: escalado + SVM
23
24 pipeline = Pipeline([
```

```

23     ('scaler', StandardScaler()),
24     ('svm', SVC(kernel='rbf', C=1.0, gamma='scale',
25                 probability=True, random_state=42))
26 ])
27 pipeline.fit(X_train, y_train)
28
29 # 3. Evaluación
30
31 y_pred = pipeline.predict(X_test)
32 etiquetas = ['Bajo', 'Medio', 'Alto', 'Crítico']
33 print(classification_report(y_test, y_pred,
34                             target_names=etiquetas))
35
36 # 4. Probabilidades para un nuevo incidente
37
38 nuevo_incidente = pd.DataFrame([
39     {'num_hosts_afectados': 3,
40     'tipo_evento_cod': 5,
41     'bytes_exfiltrados': 102400,
42     'duracion_seg': 120,
43     'privilegios_elevados': 1}
44 ])
45 proba = pipeline.predict_proba(nuevo_incidente)[0]
46 pred = pipeline.predict(nuevo_incidente)[0]
47 print(f"\nNuevo incidente Severidad predicha:
48       {etiquetas[int(pred)]}")
49 for i, p in enumerate(proba):
50     print(f" P({etiquetas[i]}) = {p:.3f}")

```

Listing 5.1: Clasificación de incidentes por severidad con SVM

5.3. Orquestación de respuestas con Python

Una vez clasificado el incidente, Python puede orquestar respuestas automáticas integrándose con APIs de plataformas de seguridad.

```

1  import requests
2  import logging
3  from dataclasses import dataclass
4  from enum import IntEnum
5
6  logging.basicConfig(
7      level=logging.INFO,
8      format='%(asctime)s %(levelname)-8s %(message)s'
9  )
10
11  class Severidad(IntEnum):
12      BAJO = 0
13      MEDIO = 1
14      ALTO = 2
15      CRITICO = 3

```

```

16
17 @dataclass
18 class Incidente:
19     id_sistema      : str
20     ip_origen       : str
21     severidad       : Severidad
22     descripcion     : str
23
24 # URL base de la plataforma SOAR (reemplazar con URL real)
25 BASE_URL = "https://security-platform/api"
26
27 def aislar_sistema(sistema_id: str) -> bool:
28     """Envía orden de aislamiento de red al sistema
29     comprometido."""
30     try:
31         resp = requests.post(
32             f"{BASE_URL}/isolate/{sistema_id}",
33             timeout=10,
34             headers={"Authorization": "Bearer <TOKEN>"}
35         )
36         resp.raise_for_status()
37         logging.info(f"Sistema {sistema_id} aislado
38             correctamente.")
39         return True
40     except requests.RequestException as e:
41         logging.error(f"Error al aislar sistema {sistema_id}:
42             {e}")
43         return False
44
45 def bloquear_ip(ip: str) -> bool:
46     """Agrega la IP al bloqueo en el firewall perimetral."""
47     try:
48         resp = requests.post(
49             f"{BASE_URL}/firewall/block",
50             json={"ip": ip},
51             timeout=10,
52         )
53         resp.raise_for_status()
54         logging.info(f"IP {ip} bloqueada en firewall.")
55         return True
56     except requests.RequestException as e:
57         logging.error(f"Error al bloquear IP {ip}: {e}")
58         return False
59
60 def notificar_equipo(incidente: Incidente) -> None:
61     """Envía alerta al equipo de respuesta (webhook)."""
62     payload = {
63         "text": (
64             f"*ALERTA SEGURIDAD*      Severidad:
65             {incidente.severidad.name}\n"
66             f"Sistema: {incidente.id_sistema}\n"
67         )
68     }

```

```

63         f"IP: {incidente.ip_origen}\n"
64         f"Descripción: {incidente.descripcion}"
65     )
66 }
67 try:
68     requests.post(f"{BASE_URL}/notify", json=payload,
69                 timeout=5)
69     logging.info("Notificación enviada al equipo.")
70 except requests.RequestException as e:
71     logging.warning(f"Fallo en notificación: {e}")
72
73 def orquestar_respuesta(incidente: Incidente) -> None:
74     """
75     Selecciona y ejecuta acciones según la severidad del
76     incidente.
77     """
78     logging.info(
79         f"[Incidente] Sistema={incidente.id_sistema} "
80         f"Severidad={incidente.severidad.name}"
81     )
82
83     if incidente.severidad == Severidad.BAJO:
84         logging.info("Acción: registrar y monitorear.")
85
86     elif incidente.severidad == Severidad.MEDIO:
87         notificar_equipo(incidente)
88
89     elif incidente.severidad == Severidad.ALTO:
90         bloquear_ip(incidente.ip_origen)
91         notificar_equipo(incidente)
92
93     elif incidente.severidad == Severidad.CRITICO:
94         aislar_sistema(incidente.id_sistema)
95         bloquear_ip(incidente.ip_origen)
96         notificar_equipo(incidente)
97
98 #     Ejemplo de uso
99
100 inc = Incidente(
101     id_sistema = "SRV-PROD-042",
102     ip_origen = "192.168.10.55",
103     severidad = Severidad.CRITICO,
104     descripcion = "Exfiltración masiva de datos detectada."
105 )
106 orquestar_respuesta(inc)

```

Listing 5.2: Motor de respuesta automática ante incidentes

Capítulo 6

Análisis de Comportamiento de Usuarios (UBA)

6.1. Fundamentos del UBA

El *User Behavior Analytics* (UBA) consiste en establecer una línea base del comportamiento normal de los usuarios y detectar desviaciones que puedan indicar cuentas comprometidas o amenazas internas.

6.2. Construcción de perfiles de usuario

```
1 import pandas as pd
2 import numpy as np
3 from scipy import stats
4
5 def construir_perfil_usuario(logs: pd.DataFrame,
6                             usuario: str) -> dict:
7     """
8     Construye un perfil estadístico del usuario a partir
9     de registros de actividad (logs).
10    """
11    datos_usuario = logs[logs['user'] == usuario]
12
13    perfil = {
14        'usuario' : usuario,
15        'hora_media_login' :
16            datos_usuario['hora_login'].mean(),
17        'hora_std_login' : datos_usuario['hora_login'].std(),
18        'ip_frecuentes' :
19            datos_usuario['ip_origen'].value_counts()
20                .head(5).index.tolist(),
21        'bytes_enviados_media':
22            datos_usuario['bytes_sent'].mean(),
23        'bytes_enviados_std' : datos_usuario['bytes_sent'].std(),
24        'accesos_fallidos_p95': datos_usuario['failed_logins']
25            .quantile(0.95),
26    }
```



```

24     return perfil
25
26 def detectar_comportamiento_anómalo(evento: dict,
27                                     perfil: dict,
28                                     umbral_z: float = 3.0) ->
29                                     list:
30
31     """
32     Compara un evento con el perfil histórico.
33     Devuelve lista de alertas generadas.
34     """
35     alertas = []
36
37     # Hora de inicio de sesión fuera del rango habitual
38
39     if perfil['hora_std_login'] > 0:
40         z_hora = abs(
41             (evento['hora_login'] - perfil['hora_media_login'])
42             / perfil['hora_std_login']
43         )
44         if z_hora > umbral_z:
45             alertas.append(
46                 f"Inicio de sesión en hora inusual "
47                 f"(z={z_hora:.2f}): {evento['hora_login']}h"
48             )
49
50     # Acceso desde IP desconocida
51
52     if evento['ip_origen'] not in perfil['ip_frecuentes']:
53         alertas.append(
54             f"Acceso desde IP no habitual: {evento['ip_origen']}"
55         )
56
57     # Volumen de datos elevado
58
59     if (perfil['bytes_enviados_std'] > 0 and
60         evento['bytes_sent'] >
61         perfil['bytes_enviados_media'] + umbral_z
62         * perfil['bytes_enviados_std']):
63         alertas.append(
64             f"Volumen de datos inusual: {evento['bytes_sent']}
65             bytes"
66         )
67
68     return alertas
69
70 # Ejemplo
71
72 logs = pd.read_csv('user_activity_logs.csv')
73 perfil_ana = construir_perfil_usuario(logs, usuario='ana.garcia')
74
75 evento_sospechoso = {

```

```

69     'hora_login' : 3.5,          # 3:30 AM
70     'ip_origen'  : '45.33.32.156',
71     'bytes_sent' : 500_000_000  # 500 MB
72 }
73
74 alertas = detectar_comportamiento_anómalo(evento_sospechoso,
75     perfil_ana)
76 if alertas:
77     print(f"[!] ALERTAS para '{perfil_ana['usuario']}'")
78     for alerta in alertas:
79         print(f"        {alerta}")
80 else:
81     print("Comportamiento dentro del perfil habitual.")

```

Listing 6.1: Modelado de comportamiento de usuario con estadísticas

Capítulo 7

Explicabilidad de Modelos de IA (XAI)

7.1. Importancia en ciberseguridad

Muchos modelos de IA, especialmente los de aprendizaje profundo, son considerados *cajas negras*. En ciberseguridad, la capacidad de explicar *por qué* un modelo tomó una decisión es fundamental para:

- Auditorías de cumplimiento (GDPR, NIS2).
- Generación de evidencia digital forense.
- Reducción de falsos positivos validados por analistas.

7.2. SHAP: SHapley Additive exPlanations

```
1 import shap
2 import pandas as pd
3 import matplotlib.pyplot as plt
4 from sklearn.ensemble import RandomForestClassifier
5 from sklearn.model_selection import train_test_split
6
7 # 1. Entrenar modelo
8
9 df = pd.read_csv('file_features.csv').dropna()
10 X, y = df.drop('label', axis=1), df['label']
11 X_train, X_test, y_train, y_test = train_test_split(
12     X, y, test_size=0.2, random_state=42
13 )
14 rf = RandomForestClassifier(n_estimators=200, random_state=42)
15 rf.fit(X_train, y_train)
16
17 # 2. Calcular valores SHAP
18
19 explainer = shap.TreeExplainer(rf)
20 shap_values = explainer.shap_values(X_test)
```

```

19
20 #           3. Gráfico de resumen

21 shap.summary_plot(
22     shap_values[1], X_test,
23     plot_type='bar',
24     show=False
25 )
26 plt.title('Importancia SHAP           Detección de Malware')
27 plt.tight_layout()
28 plt.savefig('shap_summary.png', dpi=150)
29 plt.show()
30
31 #           4. Explicar una muestra individual

32 muestra_idx = 0
33 shap.force_plot(
34     explainer.expected_value[1],
35     shap_values[1][muestra_idx, :],
36     X_test.iloc[muestra_idx, :],
37     matplotlib=True,
38     show=False
39 )
40 plt.title('Explicación individual           Muestra 0')
41 plt.tight_layout()
42 plt.savefig('shap_force_plot.png', dpi=150)
43 plt.show()

```

Listing 7.1: Explicación de predicciones con SHAP

Capítulo 8

Ataques Adversariales contra Modelos de IA

8.1. Amenazas a los modelos de IA

Los propios modelos de IA pueden ser objetivo de ataques. Un atacante puede manipular las entradas del modelo para engañarlo y evadir la detección.

Tipos de ataques adversariales

Evasión Perturbaciones mínimas en la entrada para que el modelo clasifique malware como benigno.

Envenenamiento Inyección de datos maliciosos en el conjunto de entrenamiento.

Inversión Reconstrucción de datos de entrenamiento a partir del modelo.

Extracción Réplica del modelo mediante consultas masivas.

8.2. Defensa: entrenamiento adversarial

```
1 import numpy as np
2 from sklearn.ensemble import RandomForestClassifier
3 from sklearn.model_selection import train_test_split
4 from sklearn.metrics import accuracy_score
5 import pandas as pd
6
7 def generar_ejemplos_adversariales(X: np.ndarray,
8                                   ruido_std: float = 0.05,
9                                   seed: int = 42) ->
10                                   np.ndarray:
11     """
12     Genera ejemplos adversariales añadiendo ruido gaussiano.
13     En producción se usaría FGSM, PGD, etc.
14     """
15     rng = np.random.default_rng(seed)
16     ruido = rng.normal(0, ruido_std, X.shape)
17     return np.clip(X + ruido, 0, 1) # mantener rango normalizado
```

```

17
18 #           1. Datos y modelo base

19 df = pd.read_csv('file_features.csv').dropna()
20 X, y = df.drop('label', axis=1).values, df['label'].values
21 X_train, X_test, y_train, y_test = train_test_split(
22     X, y, test_size=0.2, random_state=42
23 )
24 modelo = RandomForestClassifier(n_estimators=200,
25     random_state=42)
26 modelo.fit(X_train, y_train)
27
28 #           2. Ataque

29 X_test_adv = generar_ejemplos_adversariales(X_test,
30     ruido_std=0.1)
31 acc_normal = accuracy_score(y_test, modelo.predict(X_test))
32 acc_adversa = accuracy_score(y_test, modelo.predict(X_test_adv))
33 print(f"Precisión normal      : {acc_normal:.4f}")
34 print(f"Precisión bajo ataque : {acc_adversa:.4f}")
35
36 #           3. Defensa: reentrenamiento con ejemplos adversariales

37 X_train_robusto = np.vstack([
38     X_train,
39     generar_ejemplos_adversariales(X_train, ruido_std=0.05)
40 ])
41 y_train_robusto = np.concatenate([y_train, y_train])
42
43 modelo_robusto = RandomForestClassifier(n_estimators=200,
44     random_state=42)
45 modelo_robusto.fit(X_train_robusto, y_train_robusto)
46
47 acc_robusto_adv = accuracy_score(
48     y_test, modelo_robusto.predict(X_test_adv)
49 )
50 print(f"Precisión modelo robusto bajo ataque:
51     {acc_robusto_adv:.4f}")

```

Listing 8.1: Generación de ejemplos adversariales y defensa

Capítulo 9

Desafíos y Buenas Prácticas

9.1. Desafíos en la implementación

Cuadro 9.1: Principales desafíos y estrategias de mitigación

Desafío	Descripción	Mitigación
Calidad de los datos	Los modelos dependen de datos precisos y representativos. Datos incompletos generan falsos positivos/negativos.	Pipelines de calidad de datos; validación automática.
Interpretabilidad	Modelos de DL son "cajas negras"; dificultan auditorías y la confianza del analista.	SHAP, LIME; preferir modelos interpretables cuando sea posible.
Sesgo en datos	Datos de entrenamiento sesgados producen modelos que discriminan ciertos patrones.	Análisis de sesgo; SMOTE; revisión periódica de datasets.
Ataques adversariales	Atacantes pueden manipular entradas para evadir el modelo de IA.	Entrenamiento adversarial; monitoreo de distribución de entradas.
Privacidad de datos	Los logs pueden contener información personal sensible (GDPR, CCPA).	Anonimización; aprendizaje federado; cifrado de datos.

9.2. Buenas prácticas de desarrollo seguro

Lista de verificación de buenas prácticas

1. **Controlar versiones del modelo:** usar MLflow o DVC para registrar experimentos y versiones.
2. **Separar entornos:** no usar el mismo entorno para entrenamiento y producción.
3. **Validar entradas:** sanitizar y validar datos que entran al modelo en producción para prevenir inyección de datos.
4. **Monitorear deriva de datos (*data drift*):** usar herramientas como Evidently AI para detectar cambios en la distribución de datos.
5. **Reentrenar periódicamente:** el panorama de amenazas cambia; los modelos deben actualizarse con nuevas muestras.
6. **Aplicar XAI:** documentar las razones de cada alerta para facilitar auditorías y reducir fatiga del analista.
7. **Gestionar secretos de forma segura:** nunca incrustar tokens o contraseñas en el código; usar variables de entorno o bóvedas (Vault).

9.3. Monitoreo de deriva de datos en producción

```
1 import numpy as np
2 import pandas as pd
3 from scipy.stats import ks_2samp
4
5 def detectar_deriva(datos_referencia: pd.DataFrame,
6                   datos_produccion: pd.DataFrame,
7                   umbral_p: float = 0.05) -> dict:
8     """
9     Compara distribuciones de producción con los datos de
10     referencia
11     usando el test de Kolmogorov-Smirnov.
12     Retorna un diccionario con columnas afectadas.
13     """
14     resultados = {}
15     for columna in datos_referencia.columns:
16         stat, p_valor = ks_2samp(
17             datos_referencia[columna].dropna(),
18             datos_produccion[columna].dropna()
19         )
20         hay_deriva = p_valor < umbral_p
21         resultados[columna] = {
22             'statistic': round(stat, 4),
23             'p_value' : round(p_valor, 6),
24             'deriva'   : hay_deriva
25         }
```



```

25
26     columnas_con_deriva = [
27         c for c, r in resultados.items() if r['deriva']
28     ]
29     if columnas_con_deriva:
30         print(f"[ALERTA] Deriva detectada en:
31             {columnas_con_deriva}")
32     else:
33         print("[OK] Sin deriva significativa en los datos de
34             entrada.")
35
36     return resultados
37
38 # Ejemplo
39
40 df_ref = pd.read_csv('network_traffic_reference.csv')
41 df_prod = pd.read_csv('network_traffic_production.csv')
42 feature_cols = ['bytes_sent', 'bytes_recv', 'duration',
43                 'src_port']
44 resultado = detectar_deriva(df_ref[feature_cols],
45                             df_prod[feature_cols])

```

Listing 9.1: Detección de deriva de datos con estadístico KS

Capítulo 10

Integración Completa: Pipeline de Seguridad con IA

10.1. Arquitectura del sistema

La figura siguiente ilustra la arquitectura de un sistema de seguridad basado en IA end-to-end:

Fuentes	Ingesta	Análisis IA	Decisión	Acción
Logs de red Endpoints SIEM	Pipeline ETL Normalización Feature Eng.	Anomaly Detection Malware Detection UBA	Motor de reglas Triage SVM Severidad	Aislar sistema Bloquear IP Notificar CSIRT

10.2. Pipeline completo en Python

```
1 import pandas as pd
2 import numpy as np
3 import joblib
4 import logging
5 from sklearn.preprocessing import MinMaxScaler
6 from sklearn.ensemble import IsolationForest,
   RandomForestClassifier
7
8 logging.basicConfig(level=logging.INFO,
9                     format='%(asctime)s %(levelname)-8s
10                        %(message)s')
11
12 class PipelineSeguridad:
13     """
14     Pipeline integral de seguridad basado en IA.
15     Integra detección de anomalías y clasificación de malware.
16     """
17     FEATURE_COLS = ['bytes_sent', 'bytes_recv',
18                     'duration', 'src_port', 'dst_port']
19
```

```

20 def __init__(self):
21     self.scaler = MinMaxScaler()
22     self.anomaly_det = IsolationForest(contamination=0.05,
23                                       random_state=42,
24                                       n_jobs=-1)
25     self.malware_clf =
26         RandomForestClassifier(n_estimators=200,
27                               random_state=42,
28                               n_jobs=-1)
29     self.entrenado = False
30
31     # Entrenamiento
32
33 def entrenar(self,
34             df_trafico: pd.DataFrame,
35             df_malware: pd.DataFrame,
36             y_malware: pd.Series) -> None:
37     """Entrena ambos modelos con datos históricos."""
38     X_trafico = self.scaler.fit_transform(
39         df_trafico[self.FEATURE_COLS]
40     )
41     self.anomaly_det.fit(X_trafico)
42     logging.info("Isolation Forest entrenado.")
43
44     self.malware_clf.fit(df_malware, y_malware)
45     logging.info("Random Forest (malware) entrenado.")
46     self.entrenado = True
47
48     # Persistencia
49
50 def guardar(self, ruta: str = 'pipeline_seguridad.pkl') ->
51     None:
52     joblib.dump(self, ruta)
53     logging.info(f"Pipeline guardado en {ruta}")
54
55 @staticmethod
56 def cargar(ruta: str = 'pipeline_seguridad.pkl') ->
57     'PipelineSeguridad':
58     return joblib.load(ruta)
59
60 # Inferencia
61
62 def analizar_trafico(self, df: pd.DataFrame) -> pd.DataFrame:
63     """Detecta anomalías en tráfico de red."""
64     if not self.entrenado:
65         raise RuntimeError("El pipeline no ha sido
66                             entrenado.")
67     X = self.scaler.transform(df[self.FEATURE_COLS])
68     df = df.copy()
69     df['anomia'] = self.anomaly_det.predict(X)

```

```

63         df['score_anomalia'] =
            self.anomaly_det.decision_function(X)
64     return df
65
66     def analizar_archivo(self, caracteristicas: dict) -> dict:
67         """Clasifica un archivo como benigno o malicioso."""
68         if not self.entrenado:
69             raise RuntimeError("El pipeline no ha sido
                entrenado.")
70         X = pd.DataFrame([caracteristicas])
71         pred = self.malware_clf.predict(X)[0]
72         proba = self.malware_clf.predict_proba(X)[0]
73         return {
74             'clasificacion'      : 'malicioso' if pred == 1 else
                'benigno',
75             'prob_benigno'       : round(proba[0], 4),
76             'prob_malicioso'     : round(proba[1], 4),
77         }
78
79     # Demo
80
81 pipeline = PipelineSeguridad()
82
83 # Entrenamiento con datos reales
84 df_trafico = pd.read_csv('network_traffic.csv').dropna()
85 df_malware = pd.read_csv('file_features.csv').dropna()
86 y_malware = df_malware.pop('label')
87 pipeline.entrenar(df_trafico, df_malware, y_malware)
88 pipeline.guardar()
89
90 # Análisis en tiempo real
91 df_nuevo = pd.read_csv('network_traffic_live.csv').dropna()
92 resultado = pipeline.analizar_trafico(df_nuevo)
93 anomalias = resultado[resultado['anomia'] == -1]
94 logging.info(f"Anomalías detectadas en tiempo real:
    {len(anomalias)}")
95
96 # Análisis de un ejecutable
97 feats_nuevo_archivo = {
98     'entry_point'      : 4096,
99     'num_sections'     : 7,
100    'entropia_max'      : 7.8,
101    'num_importaciones' : 250,
102    'file_size'         : 512000,
103 }
104 res = pipeline.analizar_archivo(feats_nuevo_archivo)
105 logging.info(
106     f"Archivo => {res['clasificacion']} "
107     f"(P(mal)={res['prob_malicioso']:.4f}) "
108 )

```

Listing 10.1: Pipeline de seguridad integrado

Capítulo 11

Resumen y Perspectivas Futuras

11.1. Lo aprendido

A lo largo de esta guía hemos abordado cómo la IA transforma la ciberseguridad:

Detección avanzada de amenazas Los modelos de ML/DL analizan grandes volúmenes de datos para identificar patrones y anomalías que indicarían una posible intrusión. Los avances futuros apuntarán a modelos más sofisticados capaces de detectar amenazas persistentes avanzadas (APT).

Detección de anomalías mejorada Los sistemas basados en IA se vuelven más precisos al identificar desviaciones del comportamiento normal, reduciendo falsos positivos y mejorando la eficacia general.

Automatización de la respuesta La IA agiliza la contención, mitigación y recuperación ante incidentes, permitiendo respuestas más rápidas y consistentes.

Integración con tecnologías emergentes La convergencia de la IA con IoT y la nube impulsará soluciones de seguridad más completas para entornos distribuidos y heterogéneos.

Consideraciones éticas y de privacidad A medida que la IA se implanta en ciberseguridad, la transparencia, la equidad y la protección de datos personales serán prioritarias (NIS2, GDPR, IA Act europeo).

11.2. Hoja de ruta recomendada

1. Establecer una base de datos histórica de eventos de seguridad limpia y etiquetada.
2. Implementar un pipeline de detección de anomalías en red con Isolation Forest o autoencoders.
3. Incorporar análisis estático de malware mediante extracción de características PE y clasificadores supervisados.
4. Integrar los modelos con la plataforma SIEM/SOAR existente.
5. Añadir capa de explicabilidad (SHAP) para auditorías.

6. Establecer procesos de reentrenamiento periódico y monitoreo de deriva.
7. Evaluar robustez frente a ataques adversariales.

Conclusión

La integración de ML e IA en la automatización de la seguridad con Python está destinada a revolucionar la manera en que las organizaciones abordan la ciberseguridad. Python continuará estando a la vanguardia de esta innovación gracias a su rico ecosistema de bibliotecas, su comunidad activa y su facilidad para integrarse con infraestructuras de seguridad existentes.